

Project 0 Pitch: Presentation Prep

Which edges can a GNN trust? A Bayesian view of edge reliability

Luis Mendez CSE5825 Bayesian Machine Learning

How to use this sheet. Sections 1–2 are what you *say*: the 3-minute plan and a slide-by-slide script (say it in your own words, don’t memorize). Sections 3–9 are what you need to *know* so you can answer anything: background, the full model, a worked example, metrics, the experiment plan, related work and links to the course. Sections 10–12 prepare you for questions. This is rehearsal material, not the written Project 0 submission, which you must write yourself.

Contents

1	The pitch in one breath	2
2	Slide-by-slide script	2
3	Background you should know	3
4	The full model	4
5	Worked example: why combining evidence helps	5
6	Metrics in detail	6
7	Experiment plan	6
8	How it connects to the course	6
9	Related work (verify each before citing)	7
10	Likely questions and answers	7
11	Risks and mitigations	9
12	Delivery tips and final checklist	9

1 The pitch in one breath

GNNs trust every edge, but edges can be noisy or attacked. I treat whether each edge is real as a hidden variable and infer it with Bayes' rule: an LLM reads the two nodes' texts to give a prior, the graph structure gives a likelihood, and the GNN weights each message by the posterior. Because I inject the fake edges myself, I can check every posterior against ground truth.

Three things the audience must remember

1. **The problem:** GNNs blindly trust edges, and fake edges can be made to look plausible.
2. **The idea:** $\text{posterior} \propto \text{LLM text prior} \times \text{structural likelihood}$, for every edge.
3. **Why it is credible:** injected edges give ground truth, so every claim is checkable.

Time budget (about 3 minutes)

Slide	Content	Time
1	Title and question	15 s
2	Motivation	30 s
3	Datasets	30 s
4	Approach (Bayes' rule for every edge)	40 s
5	Three hypotheses	35 s
6	Why this is testable	15 s
7	How I'll check it, and the ask	15 s
Total		3:00

If you run long, cut slide 6 to one sentence; slide 5 already implies it. If you run short, spend the extra time on slide 4.

2 Slide-by-slide script

Slide 1: Title (15 s)

Hi, I'm Luis. My project asks a simple question about graph neural networks: which edges in a graph should the model actually trust? I want to answer it with a Bayesian model that uses an LLM to read the node texts.

Slide 2: Motivation (30 s)

- GNNs pass messages along edges and assume every neighbor is reliable evidence.
- Edges can be noisy or deliberately injected. An attacker can connect nodes whose texts *look* related, so the fake edges are hard to spot.
- Two kinds of evidence exist (structure and text), but current defenses combine them with hand-made rules and give no uncertainty.
- **Goal:** treat each edge's reliability as a hidden variable and infer it.

Transition: "To study this I need graphs with text, and a way to know which edges are fake."

Slide 3: Datasets (30 s)

- Four standard text-attributed graphs: Cora, CiteSeer, PubMed (paper citations) and WikiCS (Wikipedia CS articles).
- Each node has raw text, a topic label, and citation or hyperlink edges.
- I add fake edges between textually similar non-neighbors (5–40% of the edge count) and an LLM score for every edge, with SBERT similarity as a cheaper baseline.
- Punchline: I know exactly which edges are fake.

Transition: “Here is how the model decides which edges to trust.”

Slide 4: Approach (40 s), the most important slide

$$\underbrace{p(z_{ij} = 1 \mid s_{ij}, d_{ij})}_{\text{posterior}} \propto \underbrace{p(z_{ij} = 1 \mid s_{ij})}_{\text{prior from LLM text score}} \times \underbrace{p(d_{ij} \mid z_{ij} = 1)}_{\text{likelihood from structure}}$$

Walk left to right: **prior card** (the LLM reads both texts), **likelihood card** (do the endpoints’ representations agree?), **posterior card** (the probability the edge is real). Then point at the bottom strip: messages weighted by the posterior, a posterior on the GNN weights too, predictions with uncertainty.

In one sentence: posterior equals text prior times structural likelihood, and the GNN uses the posterior as a soft trust weight.

Slide 5: Hypotheses (35 s)

- H1 Text and structure together find fake edges.** *Test:* AUROC for fake-edge detection.
- H2 Uncertainty flags attacked nodes.** *Test:* ECE and predictive uncertainty vs. attack size.
- H3 LLMs make a better prior than embeddings.** *Test:* AUROC with LLM vs. SBERT prior, per dataset.

Slide 6: Why this is testable (15 s)

I inject the fake edges myself, so I know the true answer for every edge and can check the posteriors directly, at every attack size, not just final accuracy.

Slide 7: Checks and ask (15 s)

To trust the results I’ll first test on simulated graphs, then compare evidence sources, check calibration, and compare three posterior approximations. I’m looking for one or two teammates interested in graph learning, LLMs or Bayesian modeling. Thanks!

3 Background you should know

3.1 Graph neural networks

A graph $G = (V, E)$ has node features X (here, encoded node text) and adjacency matrix A . A graph convolutional network (GCN) layer computes

$$H^{(\ell+1)} = \sigma\left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(\ell)} W^{(\ell)}\right), \quad \tilde{A} = A + I,$$

so each node averages its neighbors’ (degree-normalized) representations and applies a learned linear map $W^{(\ell)}$. Two layers mean information travels two hops. **Task:** semi-supervised node classification. A few nodes are labeled, and the model predicts the rest (transductive setting).

3.2 Why fake edges hurt

GNNs work because of **homophily**: linked nodes usually share a label. A fake edge between nodes of different classes pulls a node’s representation toward the wrong class. Because messages travel several hops, a few edges can flip many predictions.

3.3 The attack setting

- **Candidate pool:** for each node, rank other nodes’ texts with BM25 (a classic keyword-matching relevance score from information retrieval) and keep the top non-neighbors. These pairs look textually plausible.
- **Selection:** a white-box projected gradient descent (PGD) attack relaxes “add this edge?” to a continuous variable, follows the gradient of the classification loss, then keeps the B most damaging edges.
- **Budget:** $B = 5, 10, 20, 30$ or 40% of the original edge count; edges are added at evaluation time only. The attacker cannot change node texts.

Key point. This is the hard case for text: fake edges are chosen to look textually reasonable, so the structural signal has to carry part of the load. That is why combining both matters.

3.4 Text encoders and LLM scoring

SBERT (Sentence-BERT) maps each text to a vector; similarity is the cosine between two vectors, computed independently for each text. An **LLM judge** reads both texts together in one prompt and outputs a rating of whether the link makes sense, so it can use the relation between the texts, not just topic overlap. It is slower and its scores are not automatically calibrated.

4 The full model

4.1 Variables

Symbol	Meaning	Hidden or observed
$z_{ij} \in \{0, 1\}$	edge (i, j) is real (1) or injected (0)	hidden
W	GNN weights	hidden
s_{ij}	LLM (or SBERT) score for edge (i, j)	observed
d_{ij}	distance between endpoint representations	computed
X, E	node texts and observed edge list (incl. injected)	observed
y_{train}	labels of training nodes	observed

4.2 Prior: calibrated LLM score

$$p(z_{ij} = 1 \mid s_{ij}) = \sigma(\alpha + \beta s_{ij}),$$

with α, β learned (or given a prior). This turns a raw LLM rating into a calibrated probability: if the LLM is overconfident, β shrinks. *Extension:* share α, β across datasets through a hierarchical prior.

4.3 Likelihood: structural consistency

After propagation, compare endpoint representations, e.g. $d_{ij} = \|h_i/\sqrt{\deg_i} - h_j/\sqrt{\deg_j}\|$. Model it as a two-component mixture:

$$d_{ij} \mid z_{ij} = 1 \sim \text{Gamma}(a_1, b_1) \text{ (small, tight)}, \quad d_{ij} \mid z_{ij} = 0 \sim \text{Gamma}(a_0, b_0) \text{ (larger, spread out)}.$$

4.4 Posterior for one edge

$$q_{ij} \equiv p(z_{ij} = 1 \mid s_{ij}, d_{ij}) = \frac{\pi_{ij} p(d_{ij} \mid z = 1)}{\pi_{ij} p(d_{ij} \mid z = 1) + (1 - \pi_{ij}) p(d_{ij} \mid z = 0)}, \quad \pi_{ij} = \sigma(\alpha + \beta s_{ij}).$$

4.5 Inference loop (EM-style)

1. Initialize $q_{ij} = \pi_{ij}$ (trust the prior).
2. Propagate with messages weighted by q_{ij} to get representations h .
3. Compute d_{ij} for every edge.
4. **E-step:** update q_{ij} with the formula above. **M-step:** refit a_0, b_0, a_1, b_1 (and α, β).
5. Repeat until the q_{ij} stop changing; train W on the training labels.

Alternatives worth naming: mean-field variational inference over all z_{ij} jointly, or Gibbs sampling of z (MCMC) for small graphs like Cora.

4.6 Uncertainty over the GNN weights

Approximate $p(W \mid \mathcal{D})$ with MC dropout, deep ensembles or mean-field VI, then average predictions: $p(y_v \mid \mathcal{D}) \approx \frac{1}{T} \sum_{t=1}^T p(y_v \mid X, E, q, W_t)$. Predictive uncertainty splits into two parts:

$$\underbrace{\mathbb{H}[\bar{p}(y_v)]}_{\text{total}} = \underbrace{\mathbb{E}_W \mathbb{H}[p(y_v \mid W)]}_{\text{aleatoric (data noise)}} + \underbrace{\mathbb{I}(y_v; W)}_{\text{epistemic (model uncertainty)}}.$$

H2 predicts the *epistemic* part rises on attacked nodes.

5 Worked example: why combining evidence helps

A fake edge was chosen to look textually plausible, so the LLM is fooled: $\pi_{ij} = p(z = 1 \mid s) = 0.8$. But the endpoints' representations disagree, giving a large d_{ij} with $p(d \mid z = 1) = 0.1$ and $p(d \mid z = 0) = 0.6$:

$$q_{ij} = \frac{0.8 \times 0.1}{0.8 \times 0.1 + 0.2 \times 0.6} = \frac{0.08}{0.20} = 0.40.$$

Text alone said 80% real; with structure the posterior drops to 40%, so the message is down-weighted by more than half. Conversely, a real edge between two nodes whose representations happen to differ is protected by a strong text prior.

Key point. Each source covers the other's blind spot: text catches structurally plausible fakes, structure catches textually plausible fakes. This example is your best answer to "why both?"

6 Metrics in detail

AUROC (H1, H3)

Rank all edges by $1 - q_{ij}$. AUROC is the probability that a random fake edge ranks above a random real edge. 0.5 is chance, 1.0 is perfect. It does not depend on a threshold or on the fake/real ratio.

AUPRC

Area under precision–recall; more informative when fake edges are rare (at a 5% budget only about 1 in 20 edges is fake).

Expected calibration error (H2)

Bin predictions by confidence into B bins: $ECE = \sum_{b=1}^B \frac{|B_b|}{n} |\text{acc}(B_b) - \text{conf}(B_b)|$. Show it with a reliability diagram (confidence vs. accuracy; perfect is the diagonal).

Negative log-likelihood and Brier score

Proper scoring rules that reward both accuracy and honest confidence.

Uncertainty AUROC (H2)

How well predictive uncertainty separates attacked nodes (those touching a fake edge) from clean ones.

Accuracy under attack

Node-classification accuracy at each budget, to show the method still classifies well.

7 Experiment plan

	Compare	Metric	Supported if
H1	LLM prior only; structure only; combined posterior	AUROC/AUPRC for fake edges, per budget	combined beats both on most datasets and budgets
H2	standard GNN vs. Bayesian GNN (3 approximations)	ECE, NLL, uncertainty AUROC vs. budget	Bayesian ECE grows more slowly; uncertainty flags attacked nodes
H3	LLM prior vs. SBERT prior (and BM25)	fake-edge AUROC, accuracy	LLM wins, and the gain tracks how well text predicts labels

Baselines to mention: GCN, GAT, a structure-only robust GNN (e.g. GNNGuard or RUNG-style robust aggregation), and the same model with a uniform prior. **Repetitions:** several random seeds and splits; report mean $\pm$ standard deviation. **Sanity check first:** contextual SBM graphs with planted fake edges.

8 How it connects to the course

- **Latent variable model:** z_{ij} is a discrete latent variable per edge; the likelihood is a mixture model.
- **EM:** the inference loop is EM (E-step on z , M-step on mixture and prior parameters).
- **Variational inference:** mean-field $q(z)$ and Bayes-by-backprop for W ; the ELBO is the objective.

- **MCMC:** Gibbs sampling of z on small graphs as a gold standard to compare VI against.
- **Hierarchical modeling:** sharing prior parameters across the four datasets.
- **Model checking:** posterior predictive checks and calibration.
- **Graphical models:** the graph itself is uncertain, which is a model over structure, not just over node labels.

9 Related work (verify each before citing)

- **GNN basics:** GCN (Kipf & Welling, ICLR 2017); GAT (Veličković et al., ICLR 2018); GraphSAGE (Hamilton et al., NeurIPS 2017).
- **Attacks:** Nettack (Zügner et al., KDD 2018); PGD topology attack (Xu et al., IJCAI 2019).
- **Defenses:** GNNGuard (Zhang & Zitnik, NeurIPS 2020); RGCN (Zhu et al., KDD 2019); Pro-GNN (Jin et al., KDD 2020); GCN-Jaccard (Wu et al., IJCAI 2019).
- **Bayesian/uncertain graphs:** Bayesian GCN with a random graph (Zhang et al., AAAI 2019); Graph Posterior Network (Stadler et al., NeurIPS 2021).
- **Calibration of GNNs:** Wang et al., “Be Confident!” (NeurIPS 2021).
- **Uncertainty in deep nets:** MC dropout (Gal & Ghahramani, ICML 2016); deep ensembles (Lakshminarayanan et al., NeurIPS 2017).
- **Text and LLMs on graphs:** SBERT (Reimers & Gurevych, EMNLP 2019); LLMs for text-attributed graphs, e.g. TAPE (He et al., ICLR 2024).
- **Datasets:** Sen et al. (2008) for Cora/CiteSeer/PubMed; Mernyei & Cangea (2020) for WikiCS.

10 Likely questions and answers

Q: Why make it Bayesian instead of using an existing robust GNN?

Existing defenses output a weight per edge with no uncertainty and combine text and structure by hand. The Bayesian version gives a probability per edge that I can check against ground truth, combines both sources in a principled way, and yields calibrated predictions.

Q: What is new here?

Treating edge trust as a posterior with an *LLM-based prior* and a *structural likelihood*, and evaluating the posterior itself (detection and calibration), not just accuracy.

Q: Isn't d_{ij} circular? Representations depend on the edges you are judging.

Yes, so inference alternates, exactly like EM: representations, then posteriors, then representations again. A simpler fallback computes d_{ij} from a text-only encoder, which breaks the loop.

Q: Does the EM loop converge?

EM never decreases the likelihood when the representation step is held fixed; with the alternating GNN step it is a heuristic, so I would monitor q for stability and report it.

Q: You assume edges are independent given the data. Is that reasonable?

It is a simplification (mean-field). Attackers often target one node with several edges, so edges are correlated. A natural extension is a node-level prior on how many edges are fake.

Q: Why would an LLM beat SBERT?

SBERT compares two separately computed embeddings; an LLM reads both texts together. It may not win. That is what H3 tests, and a negative result is still useful.

Q: Isn't scoring every edge with an LLM expensive?

Cora and CiteSeer have only thousands of edges. PubMed and WikiCS are larger, so I would use a small open model with cached scores, score a subset, or fall back to SBERT there.

Q: The attacker picks textually similar pairs. Doesn't that defeat the text prior?

That is deliberate: it is the hard case for text, so the structural likelihood must do real work. An attacker who adapts to the LLM itself is future work.

Q: How will you know the LLM's scores are trustworthy?

I do not assume they are. The logistic prior learns how much to trust them, and I check calibration directly: among edges the LLM calls plausible, how many are real?

Q: Why not just threshold the posterior and delete edges?

A soft weight keeps information when the model is unsure and avoids deleting real edges; a threshold also has to be tuned. I can compare both.

Q: Are the "real" edges really all real?

No. The original graphs have their own noise. Ground truth covers injected edges only, which I state as a limitation.

Q: Why MC dropout, ensembles and VI?

They are the standard cheap approximations and are known to differ in calibration; comparing them is part of the Bayesian question.

Q: Why not full MCMC?

Too slow for network weights on larger graphs, but feasible for the edge variables on Cora, where it serves as a reference for the cheaper methods.

Q: Will this scale to big graphs?

The edge posterior is per edge, so it is linear in the number of edges; the LLM scoring is the bottleneck, handled by caching or a cheaper scorer.

Q: What if hypotheses fail?

Each has an informative negative outcome: if H1 fails, one evidence source suffices; if H2 fails, Bayesian GNNs do not help calibration under attack; if H3 fails, embeddings match an LLM at lower cost.

Q: How does this relate to your research?

It builds on a research setting I work on (robust message passing on text graphs). The class project focuses on the Bayesian formulation: edge posteriors, uncertainty and calibration.

11 Risks and mitigations

Risk	Mitigation
LLM scoring too slow or costly	small open model, cache scores, SBERT on large graphs
EM loop unstable	damp updates, or compute d_{ij} from a fixed text encoder
Mixture likelihood fits poorly	posterior predictive check on d_{ij} ; try a different family
Attack code is complex	start with random and BM25-only injection, add PGD later
Results differ by dataset	that is H3's point; report per dataset

12 Delivery tips and final checklist

- Open by stating the question, not your name and course.
 - On slide 4, physically point at the three cards in order, then at the bottom strip.
 - Read each hypothesis's *title* only; let the audience read the details.
 - If asked something you don't know: "Good question, I haven't tested that yet; my plan would be ..."
 - End on the ask, and pause for questions.
- ☐ Rehearse twice with a timer; aim for 2:45.
 - ☐ Explain the slide 4 equation and the worked example without notes.
 - ☐ Confirm dataset counts and that using the research setting is fine with your advisor.
 - ☐ Write the HuskyCT submission (data plus three hypotheses) in your own words.
 - ☐ Bring the deck as both .pptx and PDF.